TASK 26: READING AND WRITING CSV DATA

More Expanded, Topic-by-Topic Explanation | Python • Pandas • Jupyter Notebook

Learning outcome: Build a complete, repeatable CSV workflow from importing raw data through quality checks, cleaning, analysis, export, and verification.

1. CSV format: structure, strengths, and limitations

CSV means Comma-Separated Values. It represents a table as text, with one record per line and a delimiter between fields. The first row often contains column headings. CSV is easy to share, version, and import into many systems. However, it normally stores one table at a time and does not preserve spreadsheet formatting, formulas as live formulas, or reliable column types.

```
Order_ID,Customer,Region,Quantity,Unit_Price\n1001,Aarav,North,1,60000
```

CSV quoting matters. A customer name such as 'Kumar, Abhishek' contains a comma and must be quoted correctly. Pandas handles common CSV quoting rules automatically.

2. Pandas, Series, and DataFrame

Pandas is a Python library for labelled data. A Series is a single labelled column; a DataFrame is a two-dimensional collection of columns with row and column labels. A DataFrame makes filtering, grouping, calculations, and file operations easier than manually parsing text.

```
import pandas as pd\ndf = pd.read_csv('raw_sales.csv')\nprint(type(df))\nprint(type(df['Revenue']))
```

The variable `df` is a conventional name for a DataFrame. `pd` is the common alias used when importing Pandas.

3. Reading CSV with pd.read_csv()

```
df = pd.read_csv('raw_sales.csv', encoding='utf-8')
```

The function opens a file, parses rows and columns, infers types where possible, and returns a DataFrame. Key arguments:

• **filepath_or_buffer:** the input path.

• **sep:** delimiter; `sep=';'` for semicolon-separated files.

• **header:** which row supplies column names; use `header=None` if no header exists.

• **names:** supply column names when needed.

• **encoding:** decode text, usually UTF-8 when known.

• **usecols:** read only required columns to reduce memory use.

• **dtype:** set known types, especially identifiers that must retain leading zeros.

• **na_values:** define source-specific missing markers.

• **parse_dates:** request date parsing for selected columns.

• **nrows:** read a sample of rows.

• **chunksize:** read a large file in smaller pieces.

```
sample = pd.read_csv('data.csv', sep=';', usecols=['Customer', 'Revenue'], nrows=100)
```

4. File paths and working directories

A relative path such as `raw_sales.csv` is interpreted relative to Python's current working directory, which may differ from the folder containing your script. A `FileNotFoundError` often means the file is not where Python is looking.

```
from pathlib import Path\nprint('Current folder:', Path.cwd())\ninput_file = Path('raw_sales.csv')\nprint('File exists:', input_file.exists())
```

Keep the input CSV in the same folder as the script when using the supplied project. Otherwise, update the path explicitly.

5. Inspecting the dataset

```
df.head()\ndf.tail()\ndf.shape\ndf.columns.tolist()\ndf.dtypes\ndf.info()\ndf.describe()\ndf.describe(include='all')
```

`head()` and `tail()` show sample records. `shape` returns (rows, columns). `columns` lists field names. `dtypes` reveals how each column was interpreted. `info()` displays non-null counts and memory use. `describe()` calculates summaries such as count, mean, standard deviation, minimum, quartiles, and maximum for numeric columns.

Inspection should happen before cleaning because it provides a baseline. If you clean first, it can be harder to determine which issues existed in the original file.

6. Data-quality profiling: missingness, uniqueness, and duplicates

```
df.isna().sum()\ndf.isna().sum().sum()\ndf.duplicated().sum()\ndf.nunique(dropna=False)
```

Missingness counts null values. Exact duplicates are repeated rows across all checked columns. Uniqueness counts distinct values. These checks answer different questions: a column can have no missing values but still contain duplicates or invalid categories.

```
df['Order_ID'].duplicated(keep=False)  # mark every row with a repeated ID
```

Only use an Order_ID uniqueness rule if each order should appear once. Some systems use one order ID for multiple line items, so repeated IDs may be valid.

7. Missing values: detect, interpret, and handle

A missing value can mean unknown, not collected, not applicable, or failed conversion. It is not automatically zero. Pandas uses missing markers such as `NaN` and `pd.NA` depending on dtype.

```
df['Customer'].isna()\ndf['Customer'] = df['Customer'].fillna('Unknown')\ndf = df.dropna(subset=['Order_ID'])
```

Common strategies include retaining missingness, filling with an explicit placeholder, imputing values using a justified method, recovering from a trusted source, or dropping rows when the missing field is essential. Document the strategy. Avoid filling numeric fields with zero unless zero is the correct business meaning.

8. Duplicates: exact rows vs duplicate keys

```
df.duplicated().sum()\ndf.drop_duplicates()\ndf['Order_ID'].duplicated().sum()\ndf.drop_duplicates(subset=['Order_ID'], keep='first')
```

The first two examples address exact duplicate rows. The latter examples check repeated IDs. Do not drop duplicate keys automatically without understanding the data model: one order can legitimately have multiple products. In this practice dataset, one row is an exact duplicate, so `drop_duplicates()` is appropriate for the exercise.

9. Cleaning text and categories

```
for col in ['Customer', 'Region', 'Product', 'Payment_Method']:\n    df[col] = df[col].astype('string').str.strip()\ndf['Region'] = df['Region
```

Leading or trailing spaces can create categories such as 'North' and ' North '. Trimming whitespace standardizes these values. Case normalization can help if categories differ only by capitalization, but apply it only when it matches the business meaning. Check allowed categories with `value_counts()`.

```
print(df['Region'].value_counts(dropna=False))
```

10. Data type conversion

```
df['Quantity'] = pd.to_numeric(df['Quantity'], errors='coerce')\ndf['Order_Date'] = pd.to_datetime(df['Order_Date'], errors='coerce')
```

Correct types enable arithmetic, sorting, and date filtering. `errors='coerce'` turns invalid strings into missing values; inspect those new missing values. For identifiers with leading zeros, a string type may be more appropriate than numeric. If a column contains currency symbols or thousands separators, clean them before conversion.

11. Date parsing

```
df['Order_Date'] = pd.to_datetime(df['Order_Date'], errors='coerce')\nprint(df['Order_Date'].min(), df['Order_Date'].max())\nprint(df[df['Orde
```

Dates loaded as strings cannot reliably support date arithmetic. Parse dates and inspect invalid entries. Be careful with ambiguous dates such as 04/05/2026, which could mean April 5 or May 4 depending on the source convention.

12. Derived columns and validation rules

```
df['Revenue'] = df['Quantity'] * df['Unit_Price']\nprint(df[['Quantity', 'Unit_Price', 'Revenue']].head())
```

Derived fields are calculated from source fields. Recalculate revenue only when the business rule really is quantity multiplied by unit price. Discounts, taxes, refunds, and currency conversions may change the formula. For production, compare the original revenue with the calculated result and investigate discrepancies rather than overwriting silently.

13. Writing CSV with to_csv()

```
df.to_csv('cleaned_sales.csv', index=False, encoding='utf-8', date_format='%Y-%m-%d')
```

`to_csv()` exports data. `index=False` prevents an extra index column. `encoding` controls text encoding; `date_format` controls readable date formatting. Other options include `sep`, `columns`, `na_rep`, and `mode`. Use a different output filename to preserve the original raw data.

14. Encoding and CSV parsing errors

- **FileNotFoundError:** check filename, path, and current working directory.
- **UnicodeDecodeError:** confirm source encoding; UTF-8 BOM files may work with `utf-8-sig`.
- **One giant column:** delimiter may be wrong.
- **ParserError:** inspect malformed quoting or inconsistent field counts.
- **Numbers stored as text:** normalize symbols and separators, then convert.
- **Dates incorrect:** confirm source format and day/month order.
- **Extra Unnamed column:** the row index may have been exported.

15. Basic analysis after cleaning

```
print('Total units:', df['Quantity'].sum())\nprint('Total revenue:', df['Revenue'].sum())\nprint(df.groupby('Region')['Revenue'].sum().sort_va
```

After removing the exact duplicate in the sample, there are 7 rows, total quantity is 11, and total revenue is ■276,000. These are sample figures. Grouping by region or product provides a compact summary and can reveal data issues or patterns.

16. Validation and export verification

```
df.to_csv('cleaned_sales.csv', index=False)\ncheck = pd.read_csv('cleaned_sales.csv')\nassert len(check) == len(df)\nassert list(check.columns)
```

Reloading the output checks that the file was written and is readable. Compare row counts, columns, key totals, and missing values. CSV may not preserve all data types, so parse dates again when needed. A robust workflow validates important business rules as well as file shape.

17. Full end-to-end code outline

```
import pandas as pd\n\ndf = pd.read_csv('raw_sales.csv')\nprint(df.head(), df.shape)\nprint(df.isna().sum())\nprint('Duplicates:', df.duplicate
```

18. Interview questions and answers

Q. How do you read a CSV file?

A. Use `pd.read_csv('file.csv')`.

Q. How do you save a DataFrame to CSV?

A. Use `df.to_csv('output.csv', index=False)`.

Q. Why is index=False useful?

A. It prevents the DataFrame index from becoming an unwanted column.

Q. How do you check null values?

A. Use `df.isna().sum()`.

Q. How do you remove exact duplicates?

A. Use `drop_duplicates()` only after confirming that the rows are duplicates.

Q. What is the difference between exact duplicates and duplicate IDs?

A. Exact duplicates match across all checked columns; duplicate IDs share a key but may be valid separate records.

Q. What does errors='coerce' do?

A. It converts invalid values to missing values during conversion.

Q. What is encoding?

A. A mapping from characters to bytes; mismatches can cause decode problems.

Q. What is the difference between read_csv and to_csv?

A. One reads a CSV into a DataFrame; the other writes a DataFrame to a CSV.

Q. Does CSV preserve data types?

A. Not fully; Pandas infers types when reading it.

Q. How do you handle missing values?

A. Choose a method based on the field meaning and business rules.

Q. How do you validate a saved file?

A. Reload it and compare shape, columns, totals, and missing values.